

CIRCUITOS DIGITALES Y MICROCONTROLADORES 2025

Facultad de Ingeniería
UNLP

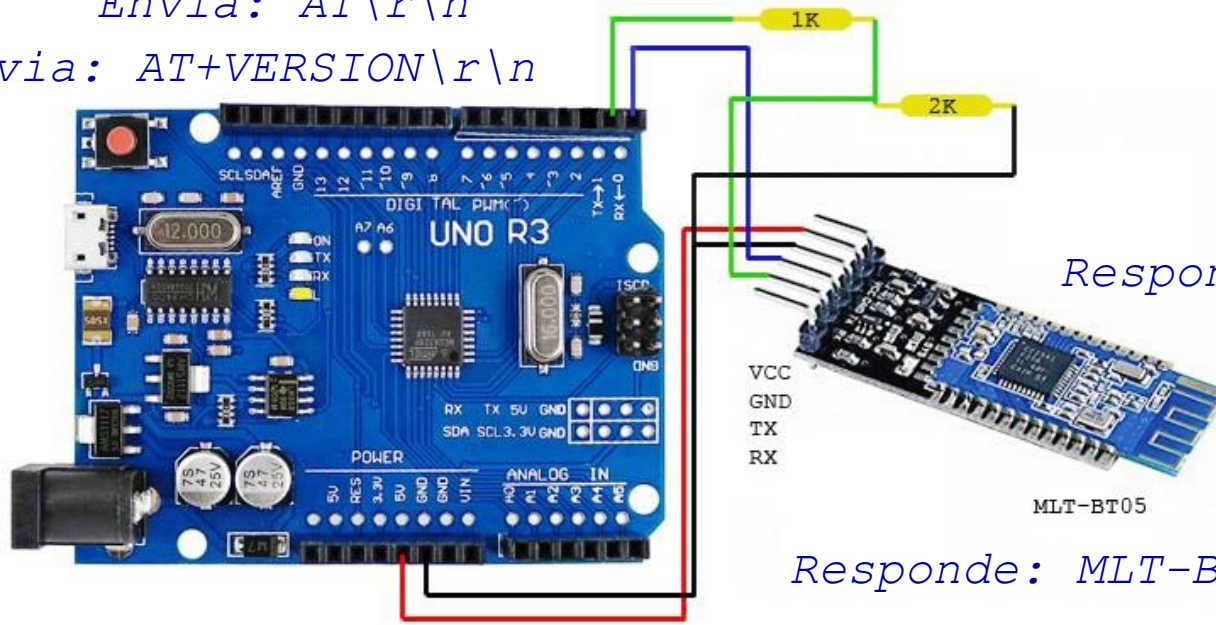
Biblioteca de funciones para el periférico
UART
(driver productor consumidor)

Ing. José Juárez

Introducción

- A continuación veremos como diseñar una biblioteca de funciones para el periférico UART para enviar y recibir mensajes de una cantidad de bytes determinada (o hasta el fin de mensaje).
- Por ejemplo, la comunicación con una terminal serie puede requerir la transmisión y recepción de cadenas de caracteres de diferentes tamaños. El usuario tipea una línea de texto y presiona <CRLF> == <ENTER>
- Otro ejemplo son los mensajes pueden representar Comandos y Respuestas entre un dispositivo maestro y un esclavo (protocolo específico).

Envia: AT\r\n
Envia: AT+VERSION\r\n



Responde: OK\r\n

Responde: MLT-BT05-V4.2\r\n

Comunicación entre tareas dentro de un programa

¿Qué sucede cuando diferentes tareas dentro de una aplicación intercambian gran cantidad de datos a diferentes velocidades?

- El problema es conocido como **Productor/Consumidor** y una solución se basa en el uso de estructuras FIFO (First In – First Out)
- Por ejemplo: las funciones de C *printf()* y *scanf()* utilizan los buffers stdin, stdout respectivamente para comunicar la aplicación de consola con el hardware de video y el teclado de la PC.
- Veremos como se aplica esta solución a los periféricos del microcontrolador y como pueden establecerse estrategias de diseño para la creación de bibliotecas de funciones que permitan controlar los mismos (Drivers).

Comunicación entre tareas dentro de un programa

- Si bien las tareas (funciones en C) pueden intercambiar información a través de sus parámetros o por variables globales, arreglos o estructuras, veremos tres casos particulares para el intercambio de datos en un contexto multitarea (cooperativo o apropiativo)
- Si tenemos en cuenta además la estrategia de encapsulación (datos privados) para modularizar el código, las tareas deberán encapsular sus datos y exponer funciones públicas para acceder a ellos (`get_val()` y `set_val()`)
- Por otro lado, las tareas pueden tener un orden de ejecución, por ejemplo, se ejecuta la tarea para recibir la respuesta por UART luego de que se ejecutó la tarea de transmitir el comando. Esto se denomina sincronización entre diferentes tareas y puede implementarse mediante distintos mecanismos del SO. Por ejemplo, la forma más simple es por banderas o flags.
- En el contexto de una arquitectura Background/Foreground una tarea puede ser un handler de interrupción y la otra una tarea de segundo plano o viceversa.
- En el contexto de la planificación Time-Triggered Cooperativa (sEOS Pont 2014) las tareas se ejecutan periódicamente y de manera cooperativa (no bloqueantes) . Si las tareas requieren de largo tiempo de ejecución se implementaran como tareas tareas muti-etapas.

Comunicación entre tareas dentro de un programa

Ejemplo:

```
int main(void)
{
    SerialPort_Init(BR9600);
    SerialPort_TX_Enable();
    while(1)
    {
        SerialPort_Send_String("Hola Mundo!\r\n");
        _delay(100);
    }
    return 0;
}
```

La tarea lee la información a enviar a través del parámetro de entrada de la función

SerialPort_Send_String(..) es una función bloqueante ¿cómo la convertimos en tarea multi-etapa?

Solución: separándola en 2 funciones no bloqueantes: una escribe el mensaje a transmitir en un Buffer y la otra lee del Buffer y transmite un byte a la vez.

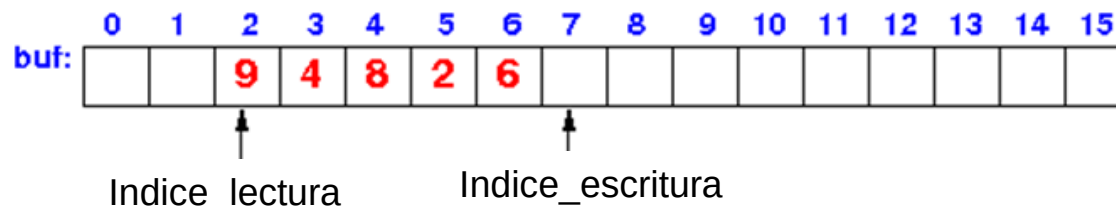
Modelo Productor/consumidor

¿qué es un buffer?

- **Buffer:** Estructura de datos de tamaño fijo, residente en RAM, que permite alojar temporalmente un conjunto de datos que poseen un determinado orden de llegada y de salida (por ejemplo FIFO). Además del arreglo o estructura de datos un buffer tiene 2 índices para su control: *un índice de lectura y otro de escritura*.

```
struct Buffer {  
    uint8_t Reloj_string[BUFFER_LEN];  
    uint8_t indice_escritura;  
    uint8_t indice_lectura;  
};
```

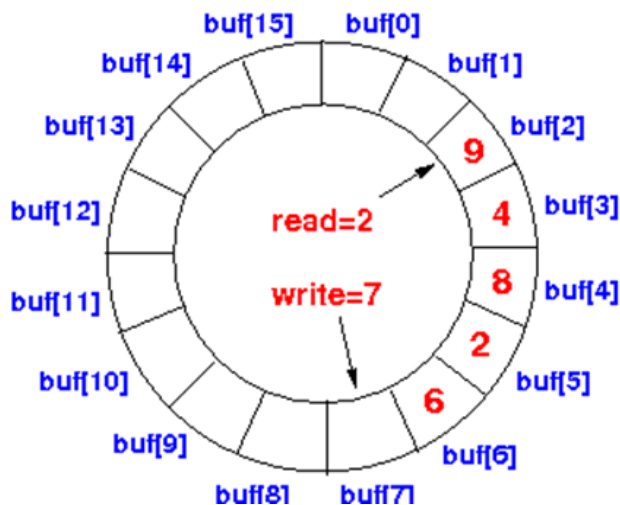
- La función que escribe en el buffer utilizará el índice de escritura. A esta función se la denomina **PRODUCTORA**
- La función que lee del buffer utilizará el índice de lectura. A esta otra se la denomina **CONSUMIDORA**



Modelo Productor/consumidor

Otras variantes básicas para el intercambio de datos entre tareas:

- **Buffer circular**: un solo buffer y dos índices, uno para leer y otro para escribir, el tamaño es fijo pero los índices recorren el mismo de manera circular sin distinguir un comienzo o un fin del mismo.



El tamaño y característica de los buffer deben seleccionarse de acuerdo a la tasa de producción y a la tasa de consumo de datos de la aplicación.

- **Buffer Ping Pong** : se utiliza cuando los datos son producidos y almacenados en grandes volúmenes para su posterior procesamiento. Un buffer es de solo escritura y el otro de solo lectura, cuando se completa un ciclo de transferencia se intercambian entre si. Ejemplo: lectura de discos, memoria de video, USB.

Modelo Productor/consumidor

- Ejemplos Típicos a los que se pueden aplicar dichos modelos:

Productor

- Teclado matricial
- Recepción UART
- Mensajes de salida
- Muestro analógico
- Reconstrucción analógica

Consumidor

- tarea que procesa texto
- tarea que procesa comandos
- tarea que transmite o imprime en display LCD
- tarea que procesa muestras
- tarea que controla el DAC

- Es común para estos casos que las funciones contengan alguno de estos nombres:

Productor

- `_Put_ (param)`
- `_Write_ (param)`
- `_send_ (param)`
- `_escribir_ (param)`
- `_enviar_ (param)`

Consumidor

- `_Get_ (¶m)`
- `_Read_ (¶m)`
- `_receive_ (¶m)`
- `_leer_ (¶m)`
- `_recibir_ (¶m)`

Arquitectura Foreground/Background

```
main (void)
{
    tarea1_Init();
    tarea2_Init();
    tarea3_Init();
```

```
sei();
```

```
while(1)
```

```
{
```

```
    if(evento_tarea1)
        tarea1();
```

```
    if(evento_tarea2)
        tarea2();
```

```
    if(evento_tarea3)
        tarea3();
```

```
}
```

```
}
```

ISR (tarea1)

```
{
```

```
...
```

```
}
```

ISR (tarea2)

```
{
```

```
...
```

```
}
```

ISR (tarea3)

```
{
```

```
...
```

```
}
```

Comunicación
por flags globales y
buffers

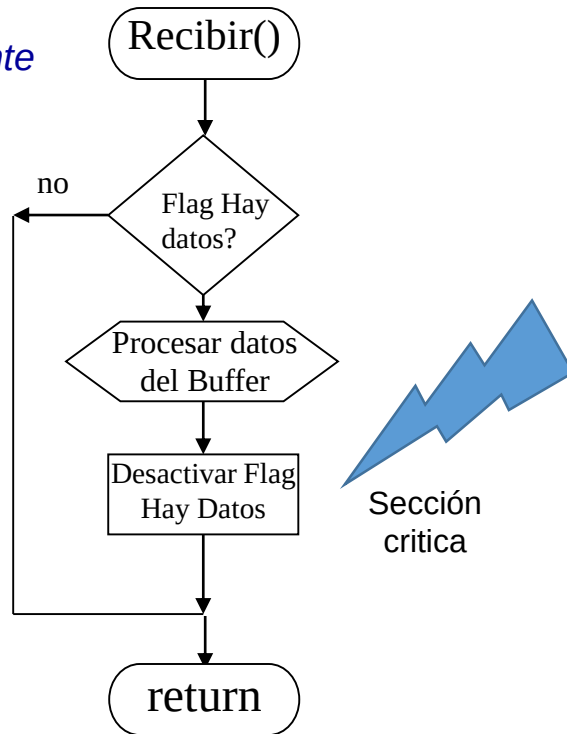
Tareas en Foreground

Tareas en Background

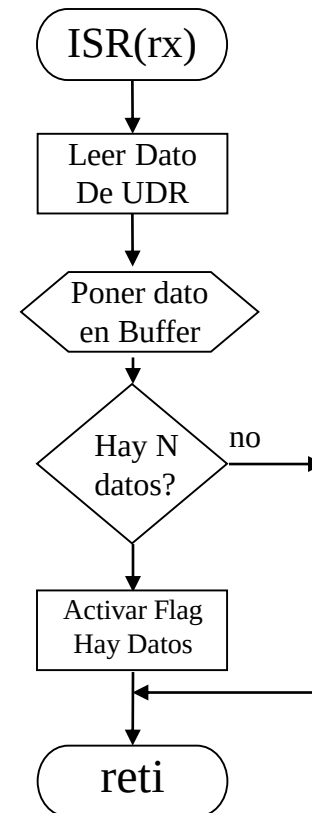
Arquitectura Foreground/Background

- Periférico serie UART. *Recepción* con buffer de tamaño [N]:

El Consumidor y el Productor se sincronizan mediante un flag!!!



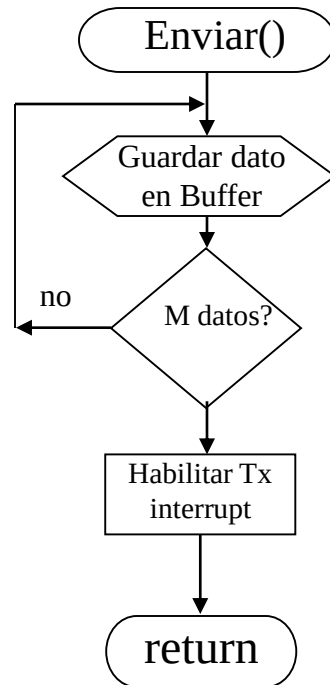
Tarea en Background- consumidor



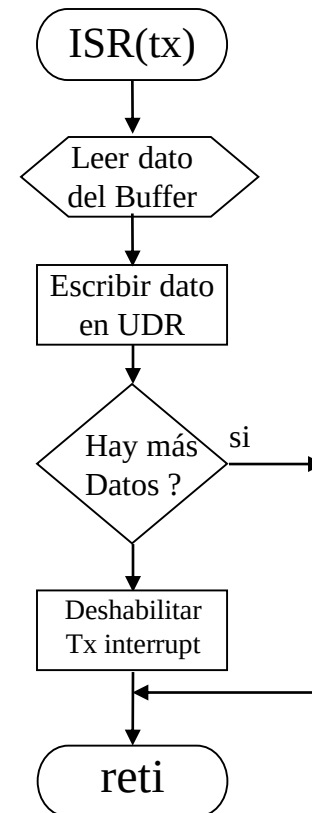
Tarea en Foreground- productor

Arquitectura Foreground/Background

- Periférico serie UART. *Transmisión* con buffer tamaño [M]:

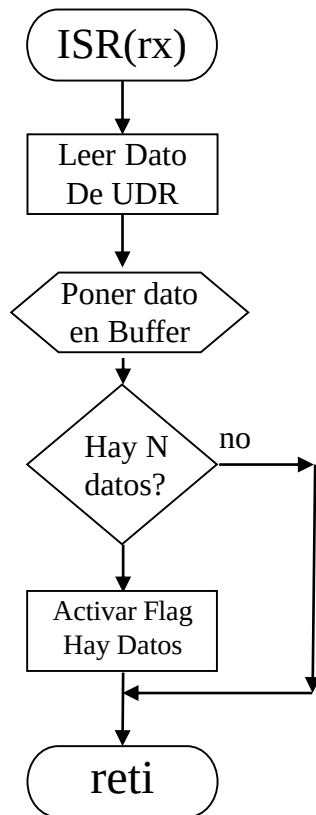


Tarea en Background - productor



Tarea en Foreground - consumidor

Ejemplo de uso de interrupción de RX

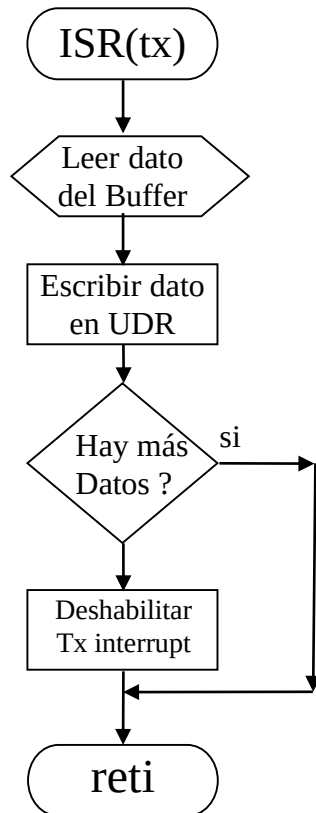


```
ISR(USART_RX_vect) {  
  
    BufferRX[Index_escritura++] = UDR0;  
    if(Index_escritura==N_DATOS){  
        FLAG_datos_recibidos=1;  
    }  
}
```

Tarea en Foreground- productor de datos de entrada

Tarea en background- consumidor de datos de entrada (hacerla...)

Ejemplo de uso de interrupción de TX



```
ISR(USART_TX_vect){ //handler de interrupción de TXC

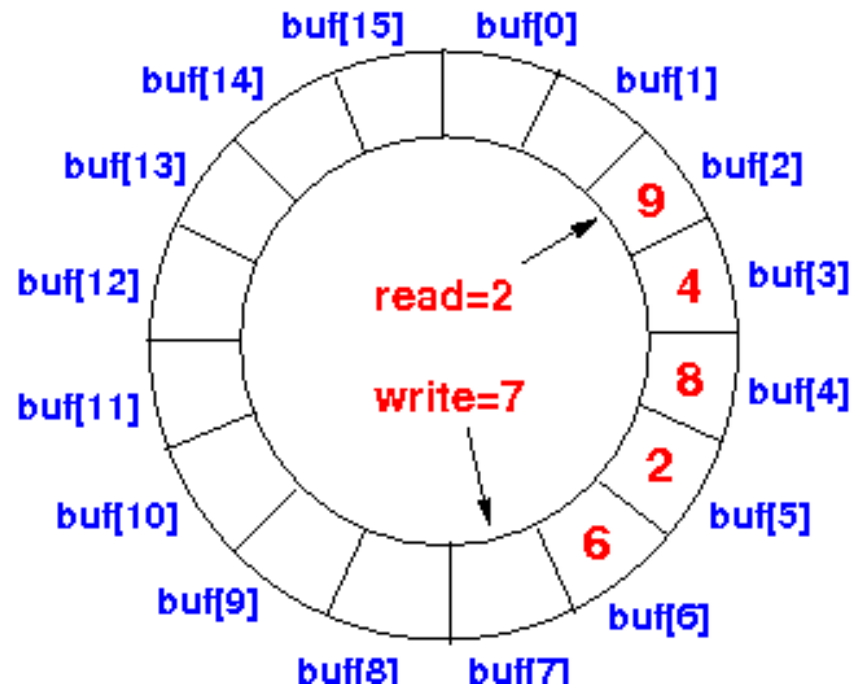
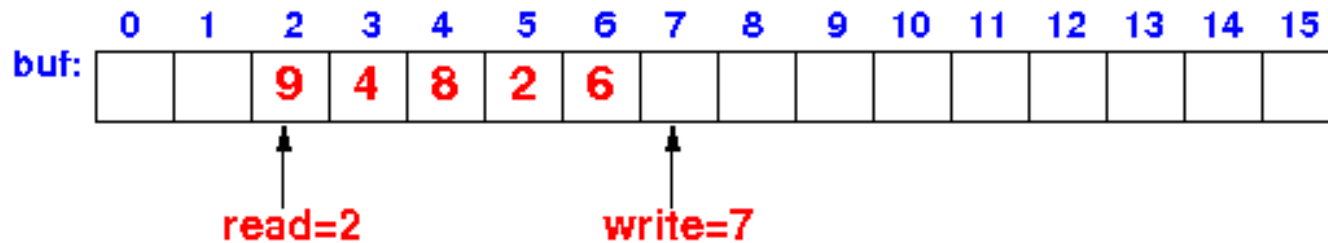
    UDR0=BufferTX[Txindex_lectura];
    Txindex_lectura++;
    if(Txindex_lectura==N_DATOS){
        Txindex_lectura=0;
        UCSRB &=~(1<<TXCIE0);
    }
}
```

Tarea en Foreground – consumidor de datos de salida

Tarea en background – productor de datos de salida (hacerla...)

Implementación con Buffer Circular

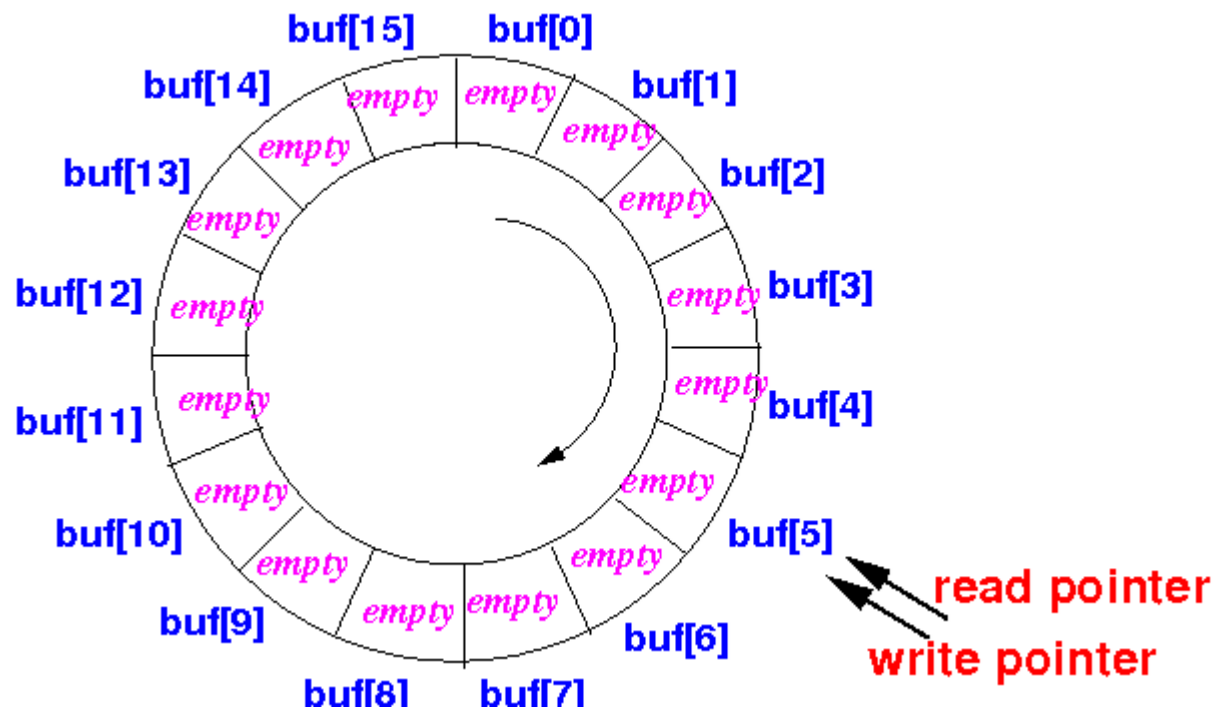
- En los ejemplos presentados se utilizó un buffer lineal, pero veremos como puede implementarse también de manera circular



Implementación con Buffer Circular

- Condición de buffer vacío:

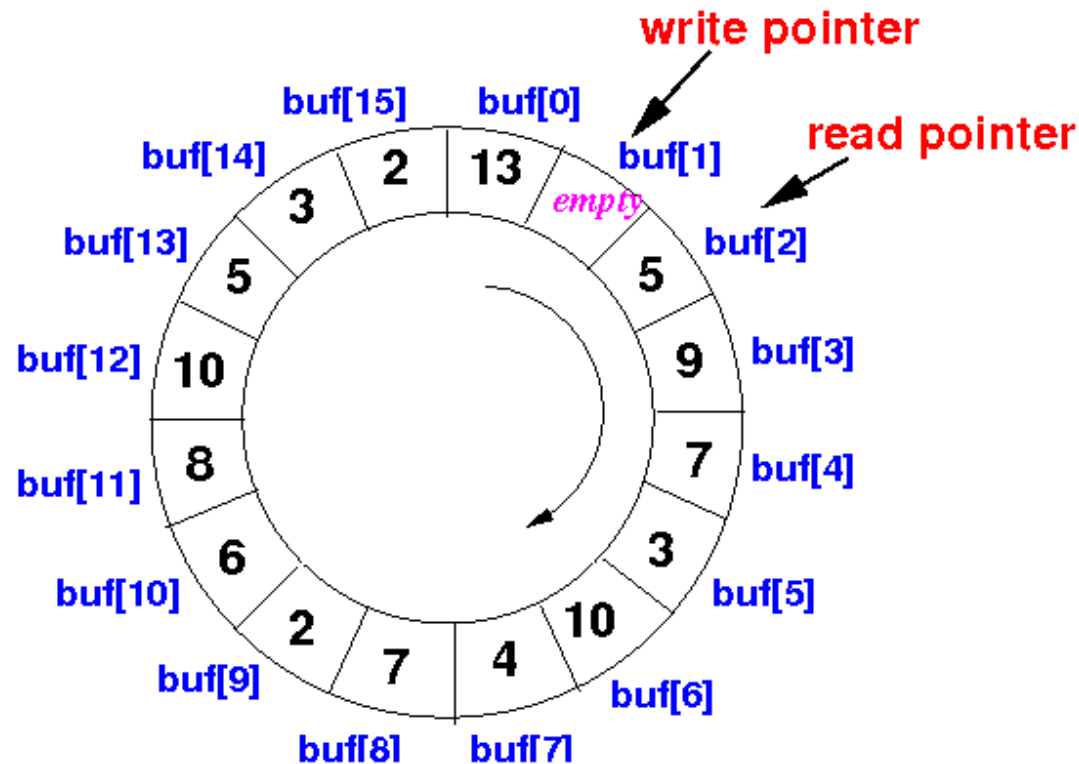
puntero de lectura = puntero de escritura



Implementación con Buffer Circular

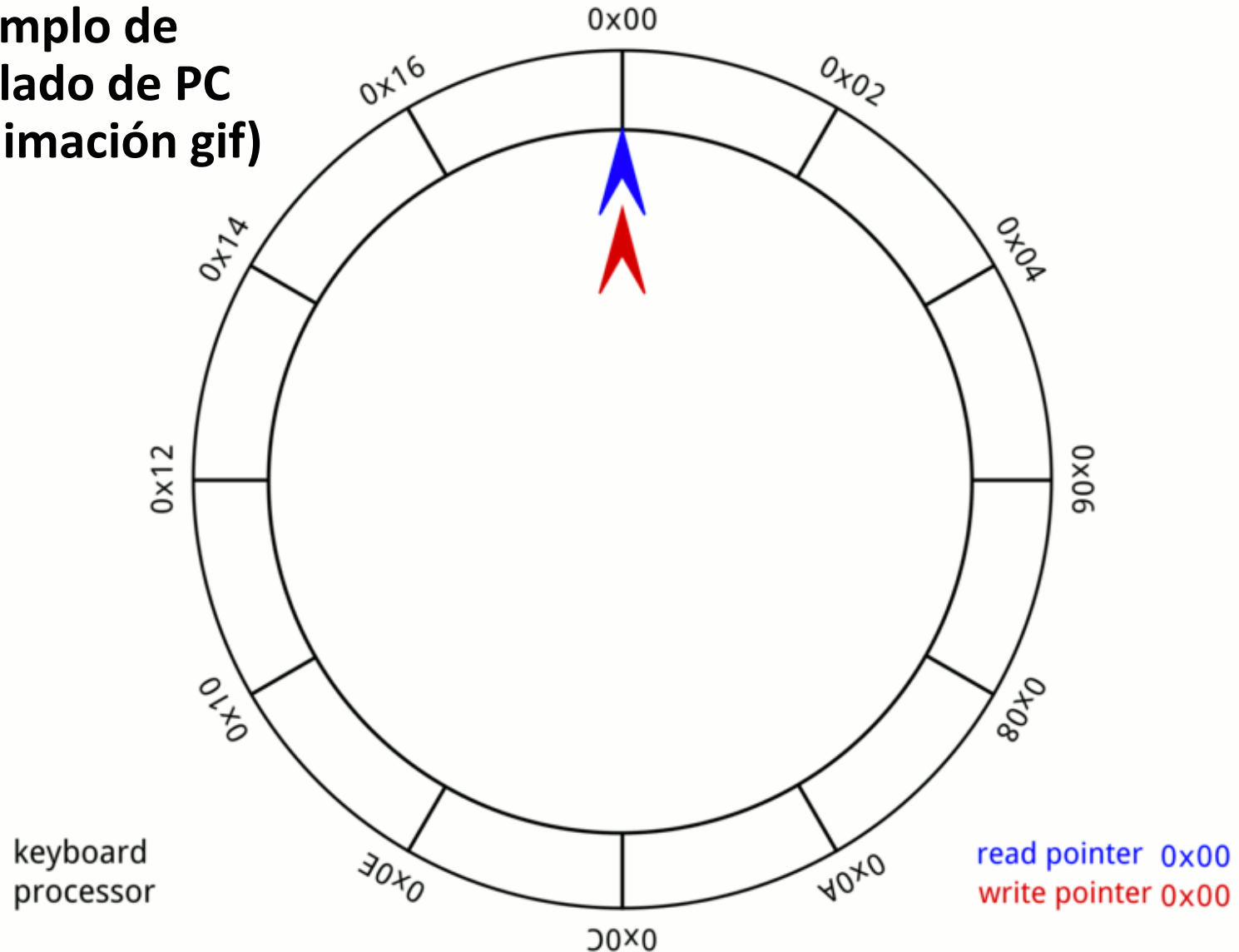
■ Condición de buffer lleno:

Asumimos buffer lleno cuando queda un solo lugar libre



Implementación con Buffer Circular

**Ejemplo de
teclado de PC
(animación gif)**



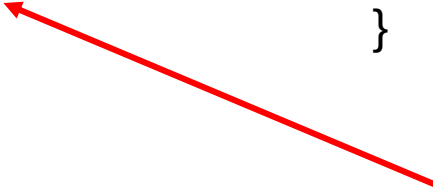
Implementación con Buffer Circular

- En código:

```
uint8 lect, esc, buffer[N];

uint8 leer (uint8 *dato)
{
    if (lect==esc)
        //Condición de vacío
        return 0;
    else {
        *dato=buffer[lect];
        lect=(lect+1)%N;
        return 1;
    }
}
```

```
uint8 escribir (uint8 dato)
{
    if( (esc+1)%N==lect )
        //Condición de Lleno (error)
        return 0;
    else {
        buffer[esc]=dato;
        esc=(esc+1)%N;
        return 1;
    }
}
```



*% da el resto de la división entera.
Con esto si $lec == N-1$, entonces da $(N-1+1)\%N=0$ y si $lec < N-1$ entonces da $lec+1$*

Bibliografía

- [1] *Embedded Microcomputer Systems, Real Time Interfacing*, Jonathan Valvano. 2007 (En biblioteca).

Los ejemplos fueron extraídos de la siguiente bibliografía:

- [2] *Patterns for time-triggered Embedded Systems*, Michael J. Pont. 2014 (descarga gratis en la web del autor)